

비동기 처리

(Asynchronous Processing)

비동기 처리

- 비동기 처리란?

- 여러 작업을 동시에 수행할 수 있는 프로그래밍 기법
- 하나의 작업이 완료될 때까지 기다리지 않고, 해당 작업이 백그라운드에서 진행되는 동안 다른 작업을 실행
- 어플리케이션의 전체 실행 시간을 단축시키고 자원 사용의 효율성을 높일 수 있음

- async와 await 이해

- `async def` : 비동기 함수를 정의하는 키워드. 이 함수는 호출 시 즉시 Future 객체를 반환하며, 이를 통해 다른 작업과 동시에 실행될 수 있음
 - Future 객체란?
 - 비동기 연산의 아직 완료되지 않은 결과를 나타냄
 - 연산의 상태를 추적하고, 완료 시 결과에 접근할 수 있게 해 줌
 - 성공, 실패 여부와 관계없이 최종 결과나 예외를 포함할 수 있음
 - 연산 완료 후 실행될 콜백을 설정하는 데 사용
- `await` : 비동기 함수 내에서 사용되며, 특정 비동기 연산이 완료될 때까지 함수의 실행을 일시적으로 중지하고, 해당 연산의 완료를 기다림

비동기 코드 예시

```
import asyncio

async def func1():
    print("func1: Start")
    await asyncio.sleep(2)  # 비동기로 2초간 대기
    print("func1: End")

async def func2():
    print("func2: Start")
    await asyncio.sleep(1)  # 비동기로 1초간 대기
    print("func2: End")

async def main():
    await asyncio.gather(func1(), func2())  # func1과 func2를 동시에 실행

if __name__ == "__main__":
    asyncio.run(main())
```

비동기 코드 예시

- 결과 해석

- func1:Start와 func2:Start가 거의 동시에 출력됨. 이는 func1과 func2가 비동기적으로 실행되기 시작했음을 나타냄
- func2는 func1보다 대기 시간이 짧기 때문에(await asyncio.sleep(1)), func2:End가 먼저 출력됨
- 이어서 func1의 대기 시간이 끝나고, func1:End가 출력됨
- 이 예시는 asyncio.gather를 사용하여 func1과 func2를 거의 동시에 실행하고 있으며, 각 함수 내에서 await asyncio.sleep(...)을 통해 비동기적으로 대기하고 있음을 보여줌. func2의 실행이 func1 보다 빨리 끝나는 것으로부터, 비동기 프로그래밍을 통해 여러 작업을 효율적으로 관리할 수 있음을 알 수 있음

asyncio 라이브러리 이해

- Python 3.4 이상에서 사용할 수 있는 비동기 I/O를 위한 표준 라이브러리
 - 이벤트 루프를 사용하여 비동기 프로그래밍을 쉽게 할 수 있도록 도와줌
 - 비동기 프로그래밍은 I/O 바운드 및 고수준 구조화된 네트워크 코드의 성능을 개선하는 데 유용
-
- 비동기 함수 정의: `async def`
 - `async def`를 사용하여 비동기 함수를 정의. 이러한 함수는 `await` 표현식을 포함할 수 있으며, 호출 시 코루틴 객체를 반환함
 - 대기하기: `await`
 - `await`는 비동기 연산이 완료될 때까지 현재 코루틴의 실행을 일시 정지 시킴. `await` 뒤에는 `awaitable` 객체가 위치해야 하며, 이는 일반적으로 `async def`로 정의된 비동기 함수의 호출임
 - 이벤트 루프
 - 이벤트 루프는 프로그램의 진입점에서 실행됨. `asyncio.run(main())`과 같이 사용하여 주어진 코루틴을 실행하고 완료될 때까지 이벤트 루프를 유지함.
 - 비동기 작업 실행 : `asyncio.gather`
 - `asyncio.gather(*tasks)`를 사용하여 여러 코루틴을 동시에 실행할 수 있음. 이 함수는 모든 코루틴이 완료될 때까지 기다린 후, 각 코루틴의 결과를 포함하는 리스트를 반환함

asyncio 라이브러리 이해

- 앞선 예제의 asyncio 중심 설명
 - func1과 func2는 각각 비동기적으로 2초와 1초 동안 대기함
 - main 함수에서 asyncio.gather를 사용하여 func1과 func2를 동시에 실행함. 이는 두 함수가 동시에 시작되게 하며, 각 함수의 실행이 서로를 방해하지 않음
 - asyncio.run(main())은 프로그램의 진입점에서 main 코루틴을 실행하고, 이벤트 루프를 관리함
- asyncio를 사용하면 복잡한 비동기 코드를 간결하고 이해하기 쉽게 작성할 수 있으며, I/O 바운드 작업의 성능을 크게 향상시킬 수 있음

FastAPI 에서의 비동기 처리 예제

```
from fastapi import FastAPI
```

```
import asyncio
```

```
app = FastAPI()
```

```
async def fetch_data():
```

```
    await asyncio.sleep(2)
```

```
    return {"data": "some_data"}
```

```
@app.get("/")
```

```
async def read_root():
```

```
    data = await fetch_data()
```

```
    return {"message": "Hello, World!", "fetched_data": data}
```

- 비동기 처리의 장점

- 비동기 처리를 사용하면, 단일 스레드에서 여러 작업을 효율적으로 수행할 수 있음
- 예를 들어, 데이터베이스 쿼리나 HTTP 요청과 같이 대기 시간이 있는 작업을 처리하는 데 유용
- FastAPI와 같은 비동기 웹 프레임워크를 사용하면, I/O 바운드 작업에 대한 성능을 크게 향상시킬 수 있음

FastAPI 에서 사용할 수 있는 비동기 기능

- FastAPI는 비동기 프로그래밍을 지원하는 기능에 대해서 비동기 기능을 지원함
 - 예) asyncio, sqlalchemy 등
- 단, sqlalchemy를 비동기 프로그램으로 사용하기 위해서는 비동기 관련 문법을 따라야 함(sqlalchemy 버전도 1.4 이상만 지원)